CNG 351 - Assignment 1

Data Management and File Structures

Fall 2026-2027

Entity-Relationship Database Designs

Team / Group ID: 8

Team member	**Student ID**
Osman Caner Cakmak	2584829
Bedri Can Guneser	2585032

Notation and model organization

There are eight drawing views. Use Case 1 has views 1A-1B, Use Case 2 has 2A-2C, and Use Case 3 has 3A-3C. An entity shown in more than one view is the same entity. Its key, attributes and relationships stay the same.

Chen notation

Symbol / marking	Meaning
Single rectangle	A strong entity with its own key.
Double rectangle	A weak entity. Its full key uses its owner's key and its own partial key.
Single / double diamond	A binary relationship / an identifying relationship.
Oval / branched oval	A simple attribute / a composite attribute and its parts.
Double oval / dashed oval	A multivalued attribute / an attribute calculated from other data.
Solid underline	The chosen key. The rules explain other unique attributes.
Dashed underline	A weak entity's partial key. BookingDate and StartTime together are Booking's partial key.
Single / double connection	Partial participation / total participation.
(min,max) beside an entity	How many times one entity instance can take part. N means many.

How to read a constraint

The numbers belong to the entity beside them. Season (0,10) means a season can have zero to ten episodes. Episode (1,1) means each episode belongs to exactly one season. The Episode side has a double line because its participation is total.

Additional rules

Each diamond connects two entity roles. An attribute on a diamond is not another entity role. We explain date limits, subset rules and totals in each use case. EmployeeCount* is calculated automatically and stored.

Use Case 1

Who Wants to Be a Millionaire?

We keep show records by season and episode. QuestionAttempt records the questions asked in a contestant's game. JokerUse records the jokers used in that game

Entity identities

Entity	Chosen full key
Season	SeasonID
Episode	EpisodeID
Host	HostID
Contestant	ContestantID
Question	QuestionID
Category	CategoryID
QuestionAttempt (weak)	(ContestantID, AttemptNo)
JokerUse (weak)	(ContestantID, UseNo)
JokerType	TypeName

Design decisions

QuestionAttempt is a weak entity owned by Contestant. AttemptNo is its partial key. It is unique only within one contestant's game and shows the question order. UsesQuestion links the attempt to the question.

JokerUse is also weak under Contestant. It records a joker's type, time and the attempt it was used for. UsedFor links it to the attempt. OfType links it to JokerType. All these relationships are binary. The four-use limit comes from Contestant's (0,4) in HasJokerUse.

ChoiceA, ChoiceB, ChoiceC and ChoiceD store the four answer choices. CorrectChoice stores the correct answer label. A question can have several categories, but it needs at least one.

For Host and Contestant, FullName has two parts: FirstName and Surname. Contestant.Age is calculated from DateOfBirth in completed years.

Use Case 1 - Rules and assumptions

Business rules and additional constraints

1. A season has no more than ten episodes. An episode belongs to one season and has one host. A contestant belongs to one episode. We keep one Contestant record per person because each person can enter the show only once.

2. HostID is the chosen host key. Host.Email is also unique. EpisodeNumber is a normal attribute. The brief does not say that it is globally unique.

3. Each question has its question text and four answer choices, A, B, C and D. CorrectChoice must be one of these labels. DifficultyLevel stores a value such as easy, medium or hard. Every question needs at least one category.

4. A game must have at least one QuestionAttempt. AttemptNo is unique within that contestant's game and increases in question order. GivenAnswer stores the chosen answer label. IsCorrect is stored and must match GivenAnswer = Question.CorrectChoice.

5. A contestant can use zero to four jokers. Each JokerUse has a UsedAt date and time, one joker type and one QuestionAttempt. The joker use and the attempt must belong to the same contestant.

6. JokerType stores Fifty-Fifty, Phone a Friend and Ask the Audience. It also stores a description of each type.

Assumptions

A1. We can add seasons, planned episodes, hosts, categories and questions before they are used. When the brief gives no minimum participation, we use zero.

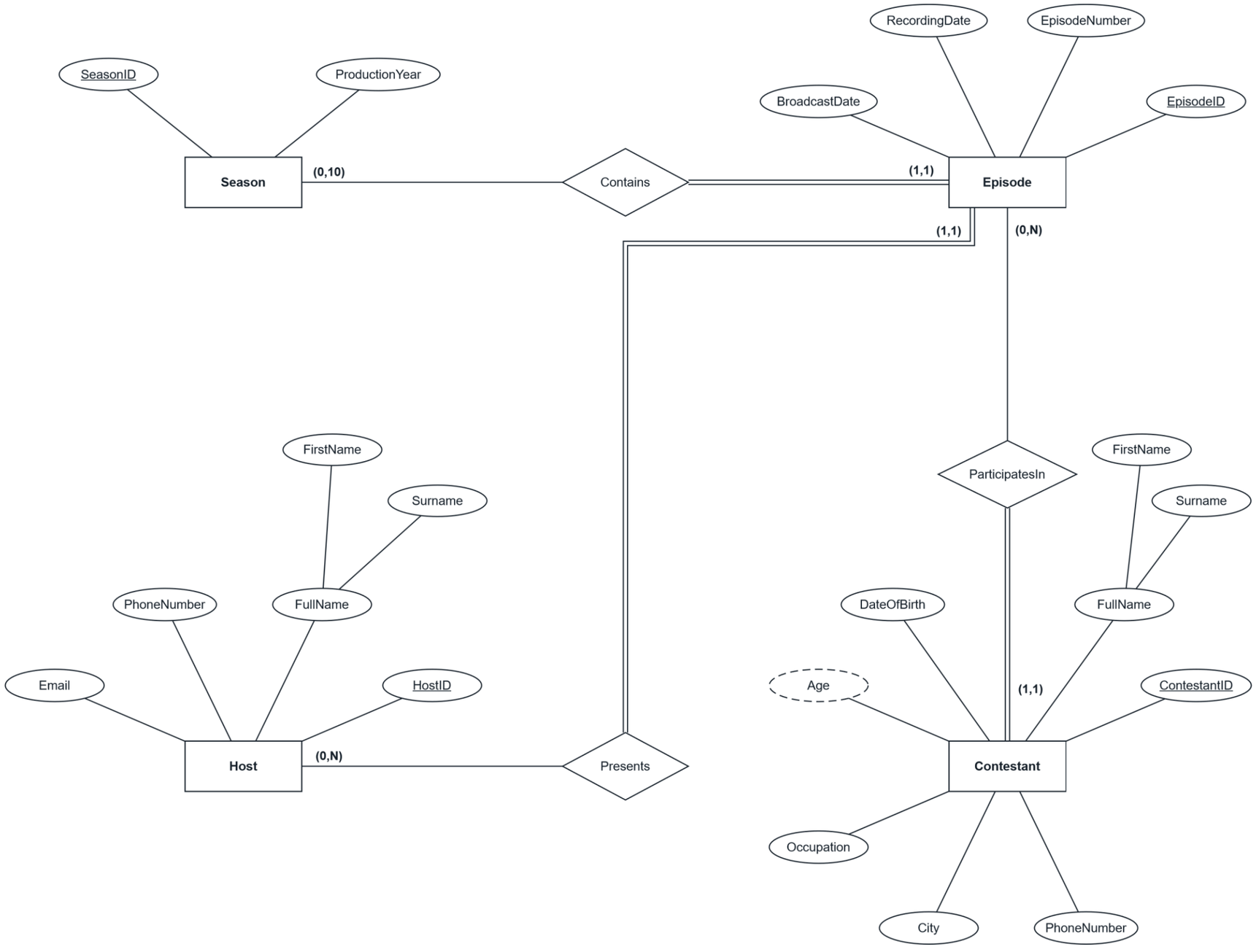
A2. The same question may be asked to the same contestant more than once. Each time has a different AttemptNo. The brief does not rule out repeated questions.

A3. A contestant may use the same joker type again. All uses count towards the total limit of four. The brief gives no separate limit for each type.

A4. Every recorded contestant has been asked at least one question. So Contestant has total participation in HasAttempt.

Use Case 1A - Production and participation

Chen ERD | Entity attributes continue in View 1B where marked. All relationships are binary.



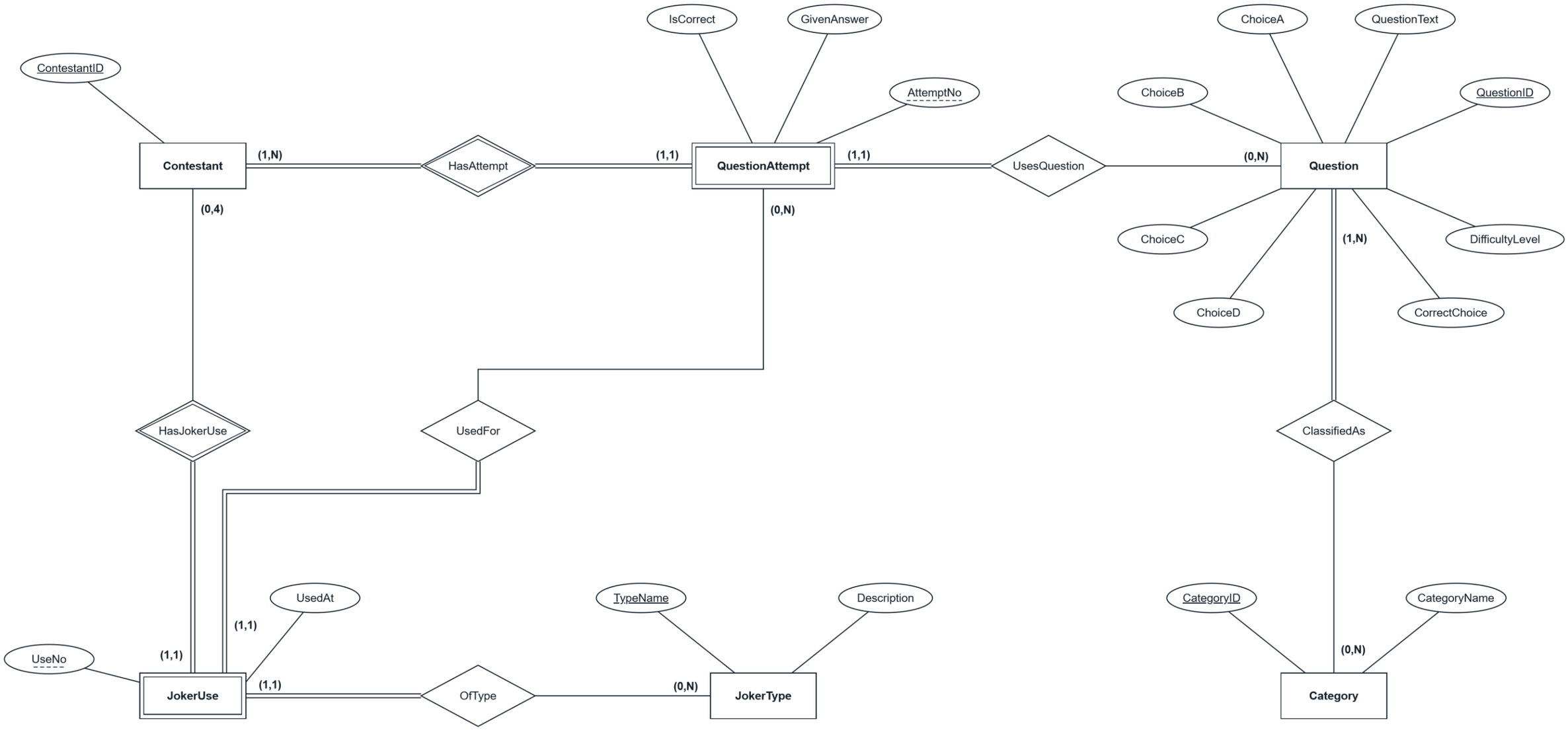
Notes

Contestant.Age is the number of completed years since DateOfBirth. Host.Email is unique in addition to HostID.

One Contestant record is kept for each person. It belongs to one episode, so a person can take part in the show only once.

Use Case 1B - Questions, answers and jokers

Chen ERD | Contestant is the same entity as in View 1A. Dashed underlines denote partial keys.



Notes

AttemptNo follows the question order within a game. The full keys are (ContestantID, AttemptNo) for QuestionAttempt and (ContestantID, UseNo) for JokerUse. UsedFor links a joker use to an attempt by the same contestant. We keep IsCorrect as a stored value and check it against GivenAnswer = Question.CorrectChoice. JokerType.TypeName is one of the three listed options: Fifty-Fifty, Phone a Friend or Ask the Audience.

Use Case 2

CleanCampus

CleanCampus has machines shared by dormitories. It records which dormitory can use each machine. It also keeps student stays, machine sessions and maintenance contracts

Entity identities

Entity	Chosen full key
LaundryRoom	RoomID
Machine	MachineID
MachineModel	ModelID
Dormitory	DormID
Student	StudentID
Stay	StayID
UsageSession (weak)	(MachineID, SessionNo)
Supplier	SupplierID
MaintenanceContract	ContractID

Design decisions

MachineModel stores the details shared by machines of the same model. Capacity has two parts, MinLoad and MaxLoad. SupportedPrograms can hold several program names, so it is multivalued. Each machine has one model. Brand stays on Machine, as the brief asks.

A dormitory's rank and free credit depend on the machine it uses. So UsageRank and FreeCreditAmount belong to the Machine-Dormitory pair in AllocatedTo. This is a binary many-to-many relationship. IsLeadingDorm is calculated from that pair's UsageRank.

Each stay has a StayID and links one Student to one Dormitory. A student who returns to the same dormitory gets a new Stay record. CurrentlyIn shows the current dormitory. It must match the open Stay.

UsageSession is weak under Machine because SessionNo is only unique within one machine. SessionNo is its partial key, and its link to Machine is identifying. MaintenanceContract has its own ContractID. This keeps a renewal separate even when the supplier and machine are the same.

Use Case 2 - Rules and assumptions

Business rules and additional constraints

1. Machine.Type is standard or industrial. Dormitory.Level is undergraduate or graduate. Student.Username is unique as well as StudentID. Student.Age is stored because no birth date is given.

2. Each machine has exactly one MachineModel and one LaundryRoom. The model stores the program names it supports. Its load limits must satisfy MinLoad <= MaxLoad.

3. A Machine-Dormitory pair has at most one allocation. It has its own UsageRank and FreeCreditAmount. IsLeadingDorm is true exactly when UsageRank = 1 for that machine.

4. Each student has one current dormitory through CurrentlyIn and at least one Stay. Exactly one Stay is open, with EndDate = NULL. Its dormitory must match CurrentlyIn. A return to the same dormitory creates a new StayID. Stay periods for the same student must not overlap.

5. SessionNo is unique within a machine. Status can be scheduled, completed, cancelled or interrupted. Duration is calculated from StartTime and EndTime when both are known.

6. A completed or interrupted session must name one dormitory. The machine must be allocated to that dormitory at the time of use. A scheduled or cancelled session may have no actual-use dormitory.

7. Each maintenance contract links one supplier and one machine. It stores StartDate, EndDate and ContractAmount. A renewal is a separate contract, even for the same supplier-machine pair.

Assumptions

A1. RoomID identifies a shared laundry room. A machine is in one room at a time. The model does not keep its earlier room locations.

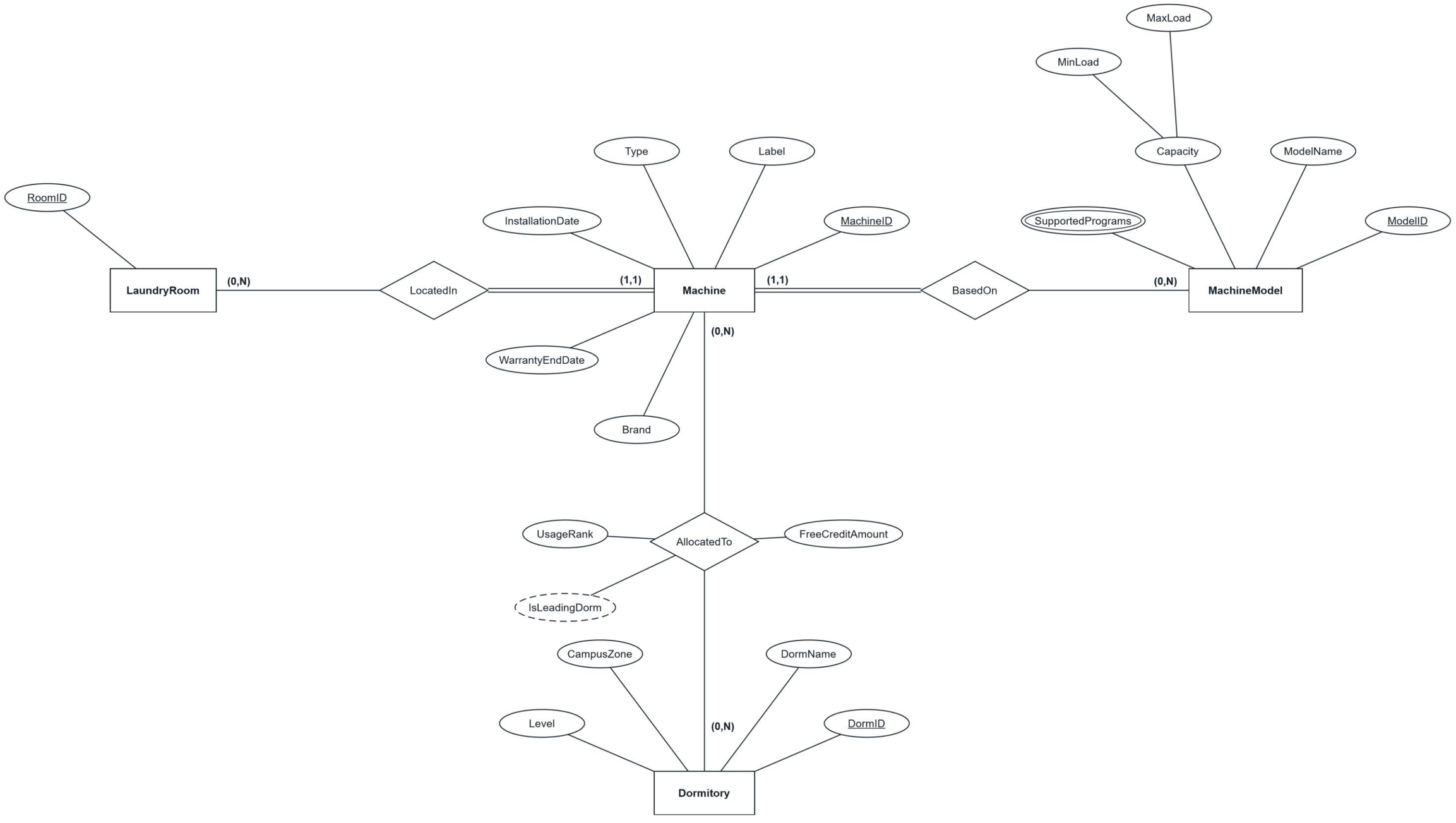
A2. Stay history includes the current open Stay. When a student leaves a dormitory, EndDate is filled in. CurrentlyIn is kept consistent with the Stay record.

A3. For each machine, UsageRank values are different positive integers. Rank 1 identifies its only leading dormitory.

A4. A session ends on its SessionDate. EndTime may be unknown for an unfinished or cancelled session. Duration is calculated only when both StartTime and EndTime are known.

Use Case 2A - Equipment and dormitory allocation

Chen ERD | Allocation attributes belong to the Machine-Dormitory pair.



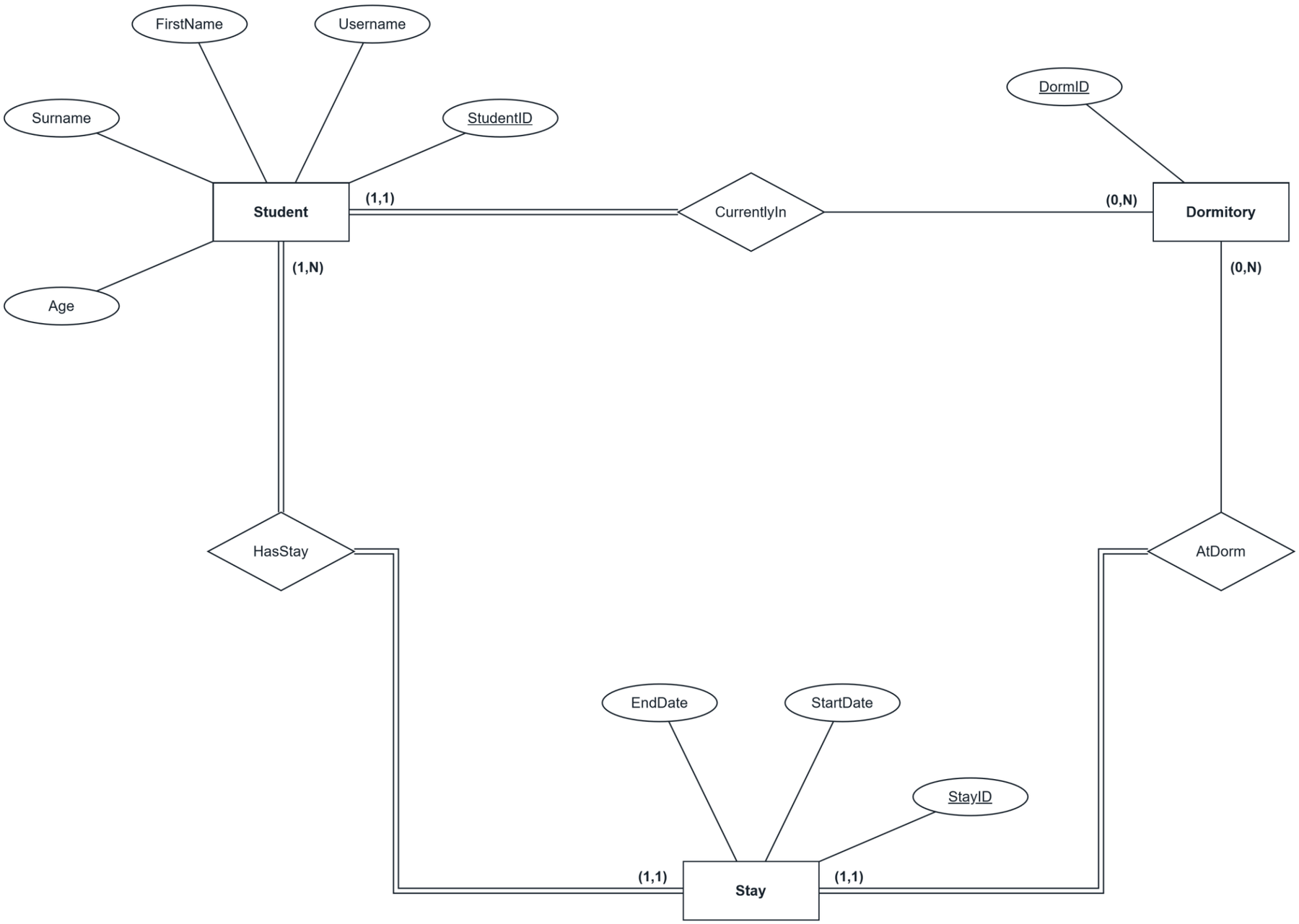
Notes

IsLeadingDorm is derived separately for each machine and is true exactly when UsageRank = 1.

Capacity is composite, with MinLoad and MaxLoad as its components. SupportedPrograms is multivalued.

Use Case 2B - Current residence and stay history

Chen ERD | Dormitory is the same entity as in View 2A. Each separate stay has its own StayID.



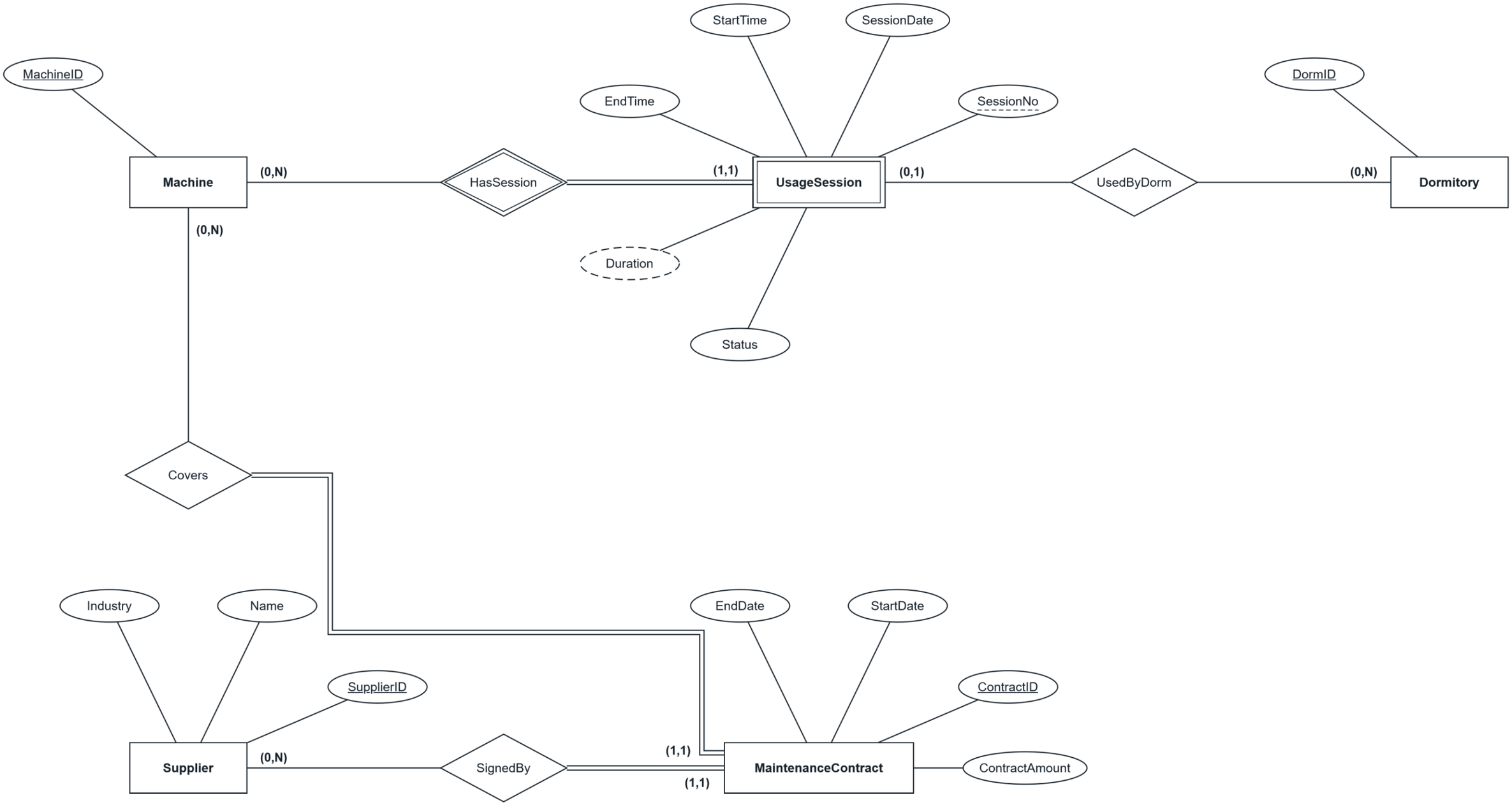
Notes

Student.Username is unique on its own, in addition to StudentID. Student.Age is stored because no birth date is given.

Each student has one open Stay with EndDate = NULL, and its dormitory must match CurrentlyIn. A return to the same dormitory gets a new StayID.

Use Case 2C - Usage sessions and maintenance

Chen ERD | Machine and Dormitory are references to View 2A. A usage session is weak under its machine.



Notes

UsageSession has the full key (MachineID, SessionNo). Duration is derived from StartTime and EndTime once both are available.
Completed and interrupted sessions must identify a dormitory. The machine must be allocated to that dormitory at the time of use.
A new maintenance agreement gets its own ContractID, even when the supplier and machine are the same.

Use Case 3

METU NCC KALTEV

The KALTEV model covers companies, their current staff and projects. It also records office rentals, internships, meeting-room bookings and investments. Rental and Investment records keep past transactions in the system.

Entity identities

Entity	Chosen full key
Company	TaxNumber
Employee	NationalID
Office	OfficeNumber
Rental	RentalID
Project	ProjectCode
Deliverable (weak)	(ProjectCode, DeliverableNo)
Intern	StudentID
MeetingRoom	RoomNumber
Booking (weak)	(RoomNumber, BookingDate, StartTime)
Investor	InvestorID
Investment	InvestmentID

Design decisions

A company may rent the same office again or receive several investments. We use separate Rental and Investment records, each with its own ID. An Investment is one transaction with a date. We use these records to calculate Company.TotalInvestmentReceived and the total for an investor-company pair.

DeliverableNo is unique only within its Project. Booking uses (BookingDate, StartTime) as a partial key within its MeetingRoom. These are weak entities. Their identifying relationships link them to their owners.

Company.Contact has two parts, Email and PhoneNumber. Employee.PhoneNumbers can have several values. CompanyAge is calculated from FoundingDate. EmployeeCount* is a stored derived value. It counts current WorksFor links and updates automatically when those links change. The asterisk means that we store the result.

Use Case 3 - Rules and assumptions

Business rules and additional constraints

1. TaxNumber identifies a Company, and NationalID identifies an Employee. Employee.Email is also unique. Each employee works for one current company through WorksFor. This relationship records EmploymentStartDate.

2. A company has one representative. An employee can represent at most one company and must currently work there. Represents is a subset of WorksFor for the same employee-company pair. Every company therefore has at least one employee.

3. A Rental links one company to one office and records StartDate, EndDate and MonthlyRent. A rental is ongoing when both date conditions hold. StartDate is today or earlier, and EndDate is NULL or later than today. A company currently operating in KALTEV needs at least one ongoing rental. We keep ended rentals as history. An office may have no ongoing rental. Rental periods for the same office cannot overlap.

4. A project belongs to one company. Its Status is proposed, active, completed or cancelled. It has at least one Deliverable. Deliverable numbering starts at D1 for each project. ActualSubmissionDate may be NULL until the deliverable is submitted.

5. WorksOn records HoursPerWeek for each employee-project pair. A project has one manager, who must also work on it. Manages is a subset of WorksOn.

6. An intern belongs to one company through InternsAt and has one supervisor. The supervisor must work for that company through WorksFor. We keep only current internships. We remove the Intern record when the internship ends.

7. There are exactly ten MeetingRoom records. A booking links one room to one company. The full key is (RoomNumber, BookingDate, StartTime). Two bookings for the same room cannot start at the same time on the same date. Bookings for different rooms can start together.

8. An Investment has one investor and one receiving company. Company.TotalInvestmentReceived adds Amount across that company's investments. The result is zero if there are none. The total from investor i to company c adds the investments linked to both i and c.

Assumptions

A1. WorksFor keeps current employment only. We do not keep records of past employment. Employees work only on projects run by their current company.

A2. Rental periods use half-open intervals [StartDate, EndDate). A NULL EndDate means there is no fixed end. A company may rent the same office again with a new RentalID.

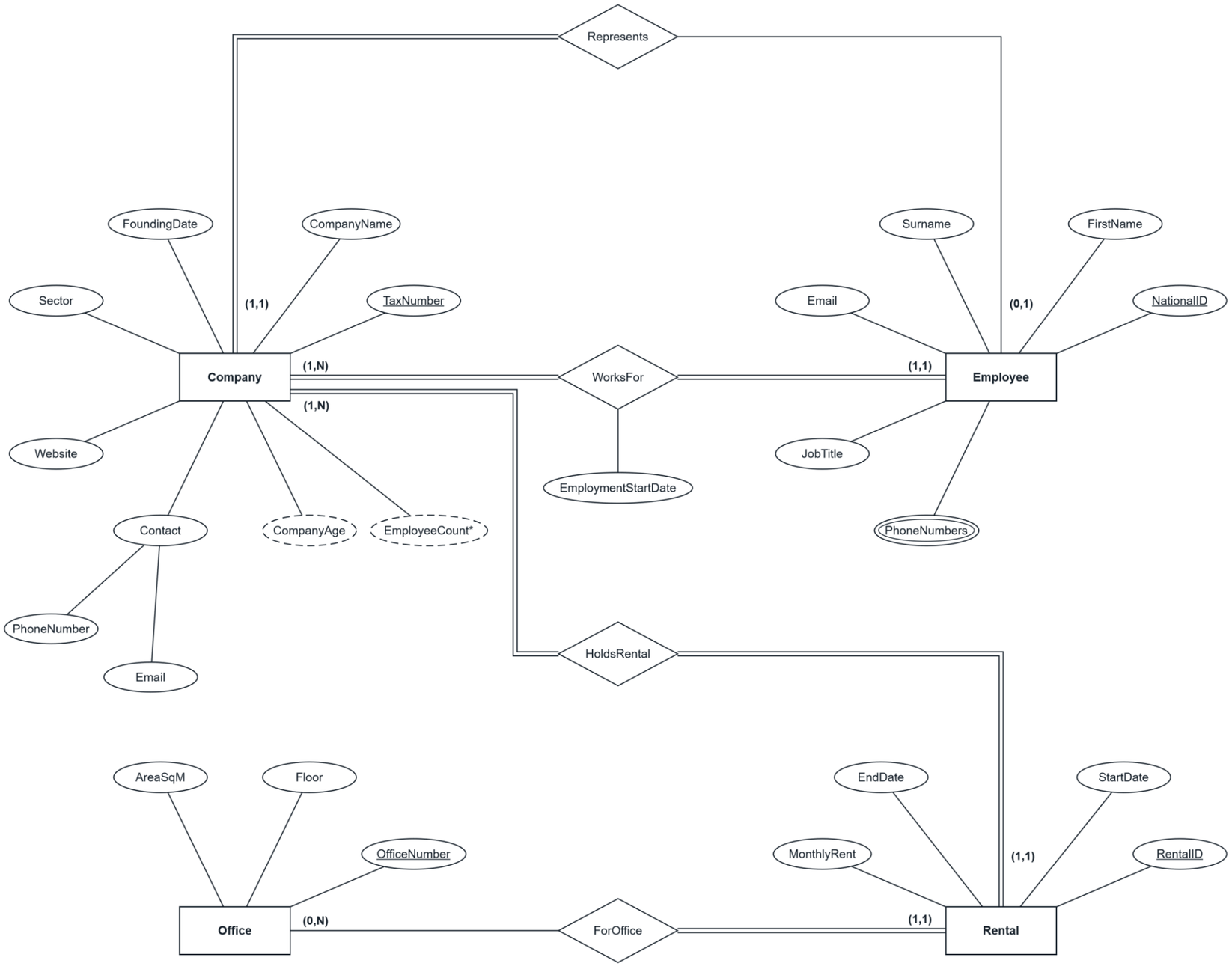
A3. A booking starts and ends on the same date. StartTime is earlier than EndTime. The key (RoomNumber, BookingDate, StartTime) stops two bookings for the same room from starting on the same date at the same time.

A4. An investor may invest in the same company more than once. Each dated transaction has a separate InvestmentID. Investor email does not have to be unique. An investor can be registered before making an investment, so Investor has partial participation in ProvidedBy.

A5. A company stays in the system after it leaves KALTEV. Its past rentals stay linked to it. The ongoing-rental rule applies only to companies currently operating in the building.

Use Case 3A - Companies, employees and offices

Chen ERD | Company.EmployeeCount* is a stored, automatically maintained derived value.



Notes

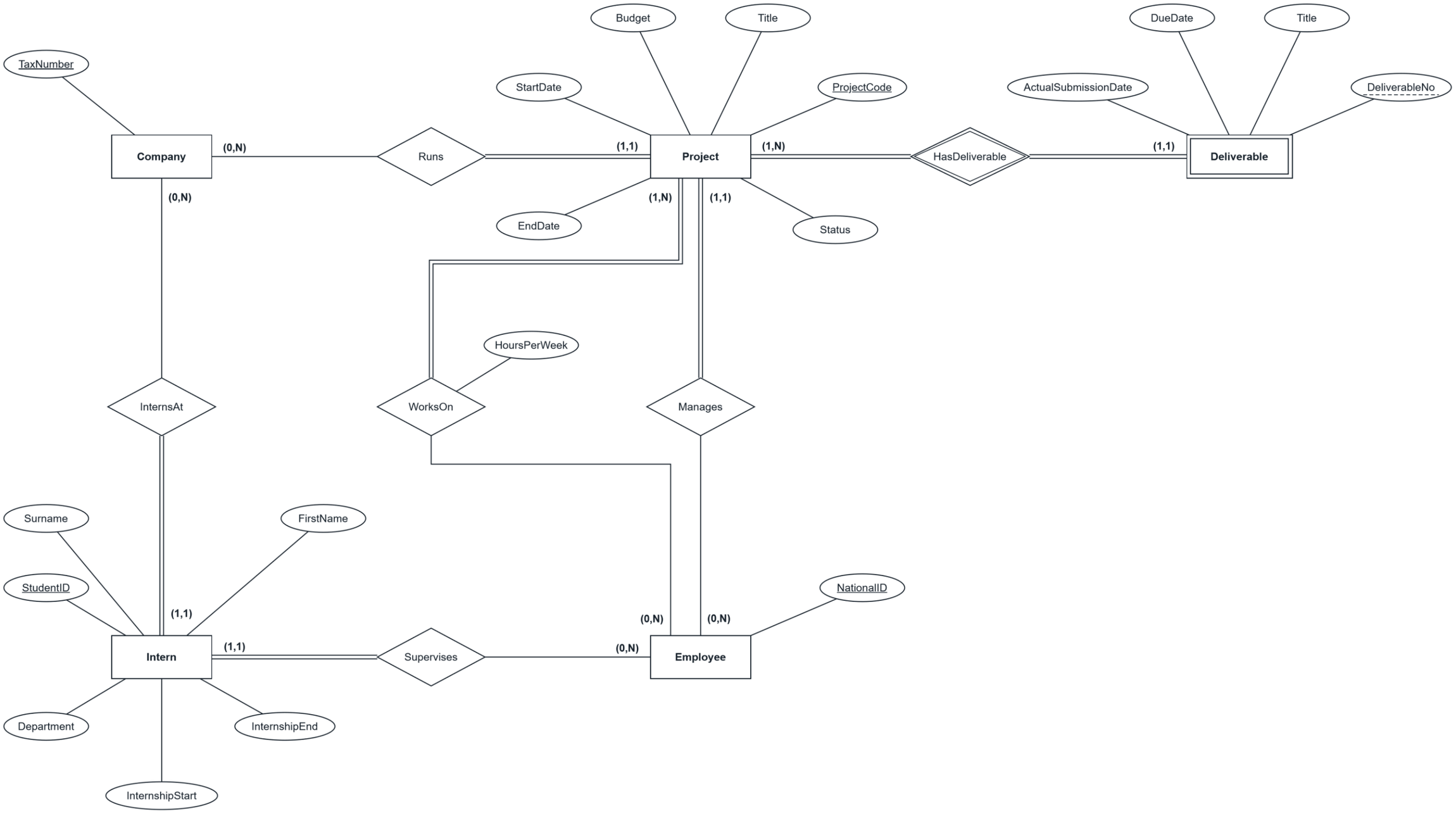
A company's representative must be one of its own current employees. Represents is therefore a subset of WorksFor.

Every company currently operating in KALTEV needs at least one ongoing rental. Rental periods for the same office must not overlap.

Employee.Email is unique in addition to NationalID. EmployeeCount* is the stored count of current employees in WorksFor and is updated automatically.

Use Case 3B - Projects, deliverables and interns

Chen ERD | Company and Employee refer to View 3A. Every project manager must also work on that project.

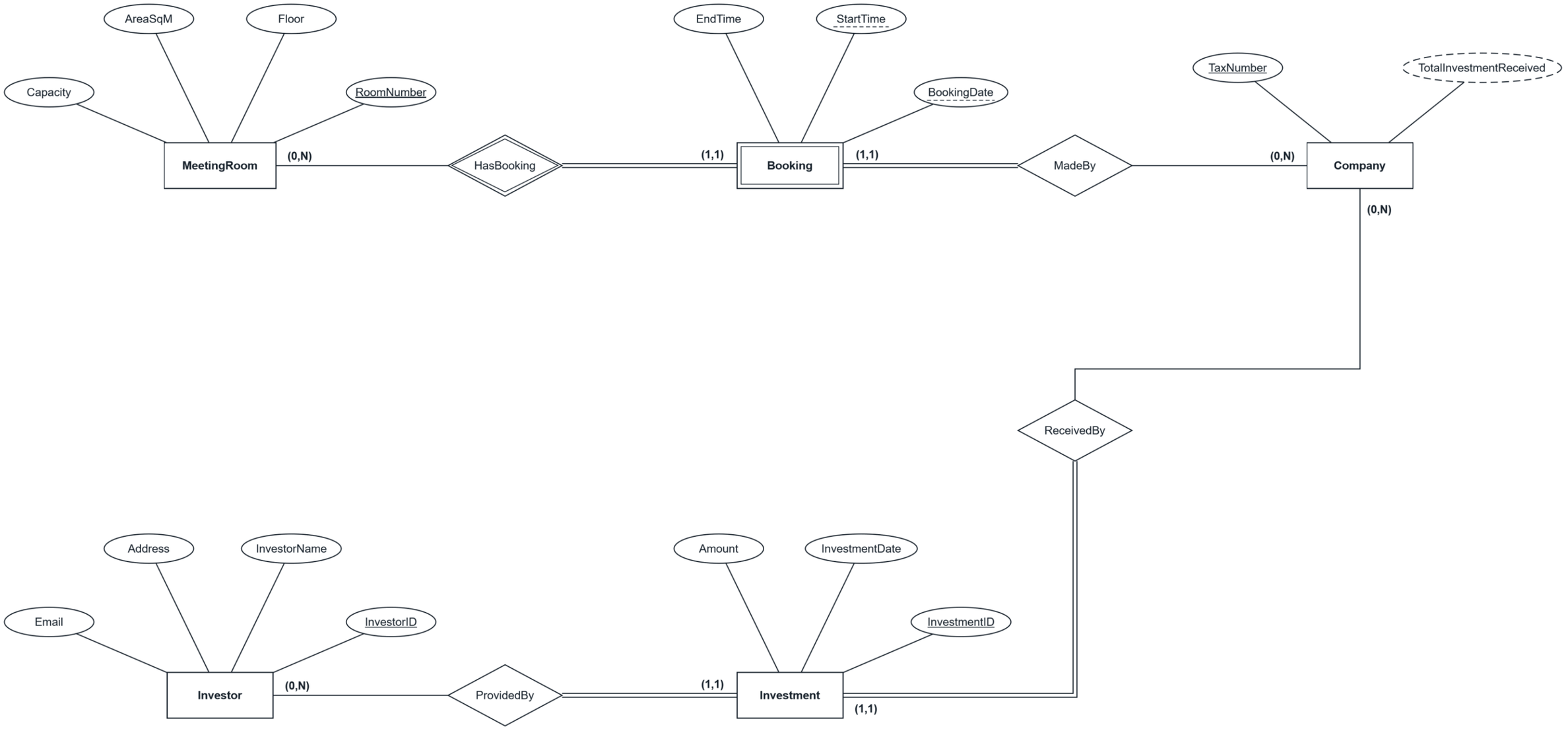


Notes

The full key for Deliverable is (ProjectCode, DeliverableNo). Numbering starts at D1 again for each project.
Manages is a subset of WorksOn: a project manager must also work on that project. All project workers belong to the company running it.
An intern and their supervisor must work for the same company. Only current internships are kept.

Use Case 3C - Meeting rooms and investment

Chen ERD | Company is the same entity as in View 3A; TotalInvestmentReceived is shown here.



Notes

Booking is identified by (RoomNumber, BookingDate, StartTime). Its partial key is (BookingDate, StartTime).

There are exactly ten MeetingRoom records. Each booking links one room to one company.

Each investment is kept as a separate transaction. Totals sum Amount for either a company or an investor-company pair.